

**Task 1:** Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions.

**Task 2:** Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

---

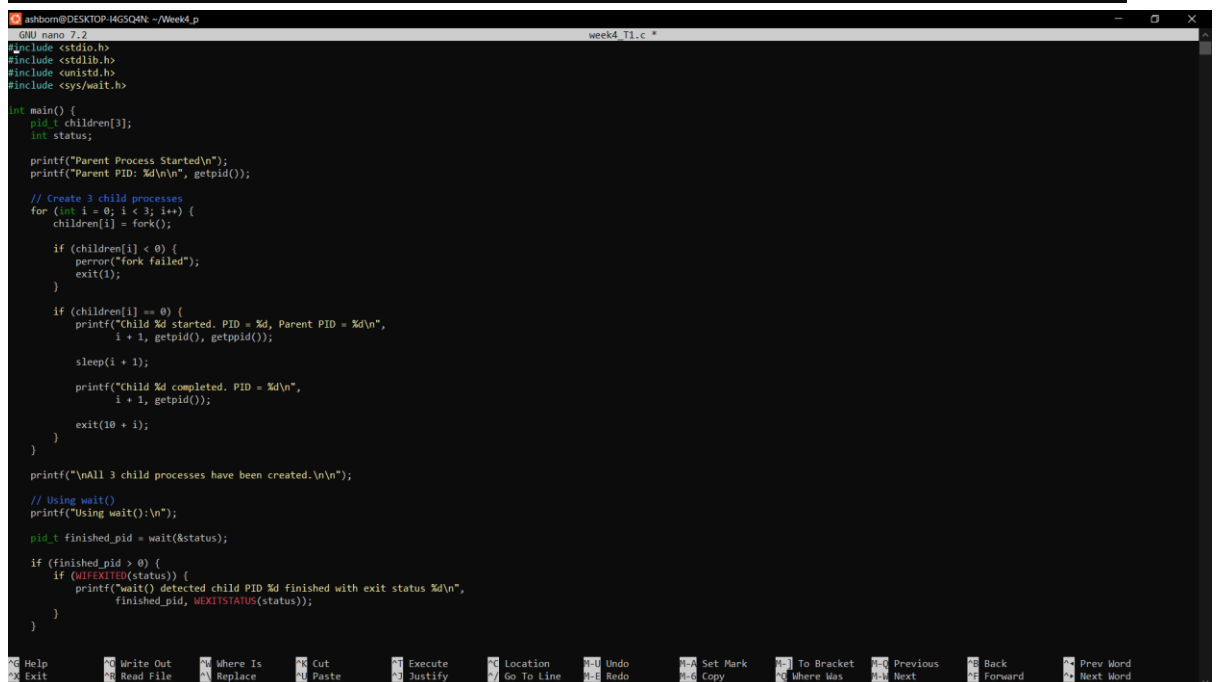
## TASK\_1

1. Created a folder for Week4 practical

```
ashborn@DESKTOP-I4G5Q4N:~$ cd ~/Week4_p
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ pwd
/home/ashborn/Week4_p
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$
```

2. Created a C-program file and compiled it

```
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ nano week4_T1.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ ls -l
total 4
-rw-r--r-- 1 ashborn ashborn 1583 Aug 17 05:15 week4_T1.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ gcc week4_T1.c -o week4_T1
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ ls -l
total 36
-rwxr-xr-x 1 ashborn ashborn 16400 Aug 17 05:18 week4_T1
-rw-r--r-- 1 ashborn ashborn 1583 Aug 17 05:15 week4_T1.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$
```



```
GNU nano 7.2 week4_T1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t children[3];
    int status;

    printf("Parent Process Started\n");
    printf("Parent PID: %d\n", getpid());

    // Create 3 child processes
    for (int i = 0; i < 3; i++) {
        children[i] = fork();

        if (children[i] < 0) {
            perror("fork failed");
            exit(1);
        }

        if (children[i] == 0) {
            printf("Child %d started. PID = %d, Parent PID = %d\n",
                i + 1, getpid(), getppid());

            sleep(1 + i);

            printf("Child %d completed. PID = %d\n",
                i + 1, getpid());

            exit(10 + i);
        }
    }

    printf("\nAll 3 child processes have been created.\n");

    // Using wait()
    printf("Using wait():\n");

    pid_t finished_pid = wait(&status);

    if (finished_pid > 0) {
        if (WIFEXITED(status)) {
            printf("wait() detected child PID %d finished with exit status %d\n",
                finished_pid, WEXITSTATUS(status));
        }
    }
}
```

3. Executed the program

```
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ ./week4_T1
Parent Process Started
Parent PID: 137

Child 1 started. PID = 138, Parent PID = 137
Child 2 started. PID = 139, Parent PID = 137

All 3 child processes have been created.
Child 3 started. PID = 140, Parent PID = 137

Using wait():
Child 1 completed. PID = 138
wait() detected child PID 138 finished with exit status 10

Using waitpid():
Child 2 completed. PID = 139
waitpid() detected child PID 139 finished with exit status 11
Child 3 completed. PID = 140
waitpid() detected child PID 140 finished with exit status 12

Parent process completed.
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$
```

Conclusion –

The program successfully created multiple child processes and demonstrated the use of both `wait()` and `waitpid()`. It showed how the parent process can synchronize and detect the completion of individual child processes.

## TASK\_2

1. Created a C-program file and compiled it

```
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ nano week4_T2_zombie.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ ls -l
total 40
-rwxr-xr-x 1 ashborn ashborn 16400 Aug 17 05:18 week4_T1
-rw-r--r-- 1 ashborn ashborn 1583 Aug 17 05:15 week4_T1.c
-rw-r--r-- 1 ashborn ashborn 597 Aug 17 05:32 week4_T2_zombie.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ gcc week4_T2_zombie.c -o week4_T2_zombie
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ ls -l
total 56
-rwxr-xr-x 1 ashborn ashborn 16400 Aug 17 05:18 week4_T1
-rw-r--r-- 1 ashborn ashborn 1583 Aug 17 05:15 week4_T1.c
-rwxr-xr-x 1 ashborn ashborn 16224 Aug 17 05:33 week4_T2_zombie
-rw-r--r-- 1 ashborn ashborn 597 Aug 17 05:32 week4_T2_zombie.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$
```

```
ashborn@DESKTOP-I4G5Q4N: ~/Week4_p
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
        printf("Child process exiting...\n");
        exit(0);
    }
    else {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child process: PID = %d\n", pid);
        printf("Parent is sleeping. The child will become a zombie.\n");

        sleep(20);

        printf("Parent process exiting.\n");
    }

    return 0;
}
```

2. Executed the program

```
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ ./week4_T2_zombie
Parent process: PID = 185
Child process: PID = 186
Child process: PID = 186
Child process exiting...
Parent is sleeping. The child will become a zombie.
Parent process exiting.
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$

ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ ps -o pid,ppid,state,stat,cmd -p 185,186
PID  PPID S  STAT CMD
185   82  S  S    ./week4_T2_zombie
186   185 Z  Z    [week4_T2_zombie] <defunct>
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$
```

3. Created another C-program and compiled

```
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ nano week4_T2_fixed.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ ls -l
total 60
-rwxr-xr-x 1 ashborn ashborn 16400 Aug 17 05:18 week4_T1
-rw-r--r-- 1 ashborn ashborn 1583 Aug 17 05:15 week4_T1.c
-rw-r--r-- 1 ashborn ashborn 864 Aug 17 05:48 week4_T2_fixed.c
-rwxr-xr-x 1 ashborn ashborn 16224 Aug 17 05:33 week4_T2_zombie
-rw-r--r-- 1 ashborn ashborn 597 Aug 17 05:32 week4_T2_zombie.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ gcc week4_T2_fixed.c -o week4_T2_fixed
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ ls -l
total 76
-rwxr-xr-x 1 ashborn ashborn 16400 Aug 17 05:18 week4_T1
-rw-r--r-- 1 ashborn ashborn 1583 Aug 17 05:15 week4_T1.c
-rwxr-xr-x 1 ashborn ashborn 16272 Aug 17 05:48 week4_T2_fixed
-rw-r--r-- 1 ashborn ashborn 864 Aug 17 05:48 week4_T2_fixed.c
-rwxr-xr-x 1 ashborn ashborn 16224 Aug 17 05:33 week4_T2_zombie
-rw-r--r-- 1 ashborn ashborn 597 Aug 17 05:32 week4_T2_zombie.c
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$
```

ashborn@DESKTOP-I4G5Q4N: ~/Week4\_p

GNU nano 7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child process: PID = %d\n", getpid());
        printf("Child process exiting...\n");
        exit(0);
    }
    else {
        printf("Parent process: PID = %d\n", getpid());
        printf("Child process: PID = %d\n", pid);

        printf("Parent is waiting for the child...\n");

        wait(&status);

        if (WIFEXITED(status)) {
            printf("Child process %d was reaped successfully.\n", pid);
            printf("Child exit status: %d\n", WEXITSTATUS(status));
        }

        printf("No zombie process remains.\n");
        printf("Parent process exiting.\n");
    }

    return 0;
}
```

4. Executed the program

```
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$ ./week4_T2_fixed
Parent process: PID = 196
Child process: PID = 197
Child process: PID = 197
Child process exiting...
Parent is waiting for the child...
Child process 197 was reaped successfully.
Child exit status: 0
No zombie process remains.
Parent process exiting.
ashborn@DESKTOP-I4G5Q4N:~/Week4_p$
```

Conclusion –

The program successfully demonstrated how a zombie process is created when a child terminates before its parent collects its status. Using `wait()` in the modified program successfully reaped the child and prevented the zombie process.